TP7 : Graphes et parcours en largeur

CPES ≪ Sciences des données, arts et cultures ≫ : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

30 janvier 2024

Implémentation d'un graphe en Python

Dans ce premier exercice, nous allons implémenter un graphe en Python, en utilisant la *programmation orientée objet*. Le code que vous écrirez pour implémenter le graphe vous servira tout au long du second semestre, au fur et à mesure que vous découvrirez de nouvelles notions sur les graphes : il est donc important de bien comprendre les tenants et aboutissants de votre code. Bien entendu, une correction sera mise en ligne afin que vous puissiez éventuellement corriger ou compléter les questions que vous n'auriez pas le temps de traiter.

La classe Sommet

Comme vu en cours, un graphe se compose de deux ensembles d'éléments que sont les *sommets* et les *arêtes*. De ce fait, pour pouvoir implémenter un graphe en Python, il va falloir également définir et manipuler ces notions. D'où la nécessité de créer une classe *Sommet*. On ne créera en revanche pas de classe *Arête* (bien qu'il pourrait être utile de le faire), car on utilisera le fait qu'en réalité l'ensemble des arêtes $E \subseteq V \times V$ peut s'écrire comme un couple d'éléments de l'ensemble des sommets.

Question 1 Définir une classe *Sommet*, ainsi que son constructeur, qui prendra en paramètres (en plus de `self`, qui, pour rappel, **doit être présent dans les paramètres de toutes les méthodes d'une classe**) les variables `id` et `label`. La variable `id` permettra d'associer à chaque sommet un identifiant unique (permettant donc de les distinguer entre eux avec certitude), et la variable `label` servira quant à elle à donner un nom ou une description à un sommet, si cela s'avérait nécéssaire plus tard.

Question 2 Ajouter, dans le code du constructeur, une ligne qui définit un attribut `voisins` comme étant une liste Python vide. Cet attribut représentera tous les sommmets accessibles depuis le sommet que l'on considère : c'est ainsi que nous représenterons les arêtes, via une *liste d'adjacence* tel que vu dans le cours.

Question 3 Définir une méthode `ajout_voisin`, prenant en paramètre une variable `nouveau_voisin`, et qui ajoutera ce nouveau voisin dans la liste des voisins du sommet *si il n'est pas déjà présent dans ladite liste*.

La classe Graphe

Nous allons maintenant créer une classe *Graphe*, en exploitant la notion de *Sommet* définie plus haut. Pour représenter le graphe, nous allons maintenir un ensemble d'objets *Sommet*. Pour ce faire, nous allons utiliser un dictionnaire Python, où les clés seront les identifiants uniques des objets *Sommet* (leur attribut `id`), et les valeurs, les objets *Sommet*. On aurait pû utiliser d'autres structures de données (liste Python, ensemble `set()`, etc), mais l'utilisation des dictionnaires permet d'accéder à un *Sommet* en $O(1)$ avec son `id`. En plus de ce dictionnaire, nous maintiendrons le nombre de sommets du graphe (n dans le cours), ainsi que le nombre d'arêtes (m dans le cours).

Question 4 Définir une classe *Graphe*, ainsi que son constructeur, qui ne prendra pas de paramètre (sauf `self`). Dans ce dernier, on créera plusieurs attributs :

- `sommets`, un dictionnaire Python vide.
- `n`, le nombre de sommets, initialement à 0.
- `m`, le nombre d'arêtes, initialement à 0 aussi.

Question 5 Définir deux méthodes `nombre_sommets` et `nombre_aretes`, retournant respectivement le nombre de sommets et le nombre d'arêtes de l'objet *Graphe*.

Question 6 Définir une méthode `ajout_sommet`, prenant en paramètre une variable `label`, ayant pour *valeur par défaut* une chaîne de caractère vide (`""`) ainsi qu'une variable `classe_sommet`, qui représente quel type de sommet on va ajouter au graphe. Cette méthode devra instancier un nouvel objet `classe_sommet` qu'on ajoutera au dictionnaire `sommets`. Afin de construire un objet `classe_sommet`, on écrira `nouveau_sommet = classe_sommet(...)`, en précisant ses paramètres, à savoir (a minima) label (déjà passé en paramètre à la méthode), et on utilisera le nombre de sommets pour l'`id` du sommet (ainsi, le premier sommet ajouté aura pour `id` la valeur 0, le second 1, etc). On s'assurera de mettre à jour le nombre de sommets en conséquence. *Note : L'utilisation de `classe_sommet` peut paraître étrange ici, alors qu'on a défini juste avant une classe Sommet. Cela s'explique par le fait que l'on va implémenter d'autres types de sommets qui seront des classes héritantes de Sommet, incluant des attributs spécifiques à différents algorithmes de parcours pour le graphe.*

Question 7 Définir une méthode `ajout_arete`, qui permet de créer une arête entre deux sommets déjà existants. Pour ce faire, elle prendra en paramètre deux variables `id_sommet_1` et `id_sommet_2`, respectivement les `id` des deux sommets. Afin de créer les arêtes, il suffira d'ajouter l'`id` du sommet numéro 2 dans la liste des voisins du sommet numéro 1, et inversement. On s'assurera de mettre à jour le nombre d'arêtes du graphe en conséquence.

Question 8 Définir une méthode `voisinage_sommet`, qui retourne la liste des voisins d'un sommet donné. Pour ce faire, la méthode prend en paramètre une variable `id_sommet`, représentant l'`id` du sommet dont on veut retourner la liste des voisins.

Question 9 En ajoutant à votre classe *Graphe* les deux méthodes fournies dans `code_fourni.py` (`__str__()` et `__getitem__()`), créer un graphe vide et l'afficher. Ensuite, ajouter des sommets et des arêtes afin de créer le graphe K_4, et l'afficher ensuite.

Implémentation du parcours en largeur

Enfin, nous allons implémenter l'algorithme de *parcours en largeur*, en anglais *breadth-first search (BFS)*. Comme indiqué dans le cours, afin d'implémenter cet algorithme, il faut doter les *Sommet* de trois attributs supplémentaires : une couleur, qui représente le statut du sommet dans le parcours (*blanc* pour un sommet qui n'a jamais été visité, *gris* pour un sommet qui a été visité dont au moins un des voisins est encore *blanc*, et *noir* pour un sommet déjà visité qui n'a pas ou plus de voisins *blancs*) ; une distance, qui représente la distance du sommet à l'origine du parcours, à savoir, le nombre d'arêtes qu'il a fallu suivre pour atteindre ce sommet ; et la parentalité du sommet, à savoir, à partir de quel sommet nous avons atteint le sommet courant.

Question 10 Compléter le code du constructeur de la nouvelle classe *SommetBFS*, héritante de la classe *Sommet*, disponible dans `code_fourni.py`. Il vous faut ajouter trois lignes dans ce constructeur, pour initialiser les trois attributs `couleur`, `distance` et `parent`. Pour réprésenter la valeur ∞, on utilisera `math.inf` de la bibliothèque `math`.

Question 11 Créer une méthode `initialisation_bfs` pour la classe *SommetBFS*, qui initialise les attributs définis par *SommetBFS* mais pas par *Sommet*, à savoir `couleur`, `distance` et `parent`. Cette fonction sera utilisée sur tous les sommets d'un graphe avant l'exécution de l'algorithme de parcours en largeur.

Question 12 A partir du pseudo-code de l'algorithme de parcours en largeur fourni dans le cours à la slide 42, écrire une fonction `parcours_en_largeur`, prenant en paramètres un objet *Graphe*, nommé `graphe`, qu'on supposera constitué de sommets *SommetBFS* (donc, on supposera que l'on aura défini le paramètre `classe_sommet = SommetBFS` lors de la construction du graphe) ; ainsi qu'une variable `origine`, qui représente l'`id` du *SommetBFS* à partir duquel on commence le parcours. Quelques indications pour l'écriture de cette fonction :

- Afin d'implémenter la *file* nécessaire à l'écriture de cette fonction, vous utiliserez la classe `deque` de la bibliothèque `collections`. Pour rappel, deux méthodes principales permettent de manipuler un objet `deque` en tant que *file* : la méthode `appendleft()`, qui permets d'ajouter un élément à gauche de la file, et la méthode `pop()`, qui renvoie l'élément de droite de la file **et le supprime**.
- Pensez à utiliser les méthodes déjà définies dans la classe *Graphe* afin d'implémenter la fonction. En particulier, vous aurez probablement besoin de faire appel aux méthodes `voisinage_sommet()` et `__getitem__()` (qui s'utilise avec des `[]` : par exemple pour accéder au sommet d'id `id_sommet` dans un objet de *Graphe* appelé `g`, il suffira de faire `g[id_sommet]`, et non `g.__getitem__(id_sommet)`).

Question 13 Une fois la fonction `parcours_en_largeur` implémentée, la compléter pour pouvoir observer les différentes étapes du parcours. Il vous faudra pour cela, à chaque itération du parcours, afficher : le numéro de l'itération, l'`id` du sommet visité, la liste des sommets éventuellement ajoutés à la *file* en conséquence de la visite du sommet numéro `id`, et enfin l'état de la file à la fin de l'itération en cours. Testez ensuite votre fonction sur la graphe K_4, et sur un graphe généré aléatoirement si le temps le permets (le code permettant de générer un tel graphe est présent dans `code_fourni.py`).